

Using the exact state space of a Markov model to compute approximate stationary measures*

Andrew S. Miner
Dept. of Computer Science
College of William and Mary
asminer@cs.wm.edu

Gianfranco Ciardo
Dept. of Computer Science
College of William and Mary
ciardo@cs.wm.edu

Susanna Donatelli
Dipartimento di Informatica
Universita' di Torino
susi@di.unito.it

ABSTRACT

We present a new approximation algorithm based on an exact representation of the state space $\mathcal{S}$, using decision diagrams, and of the transition rate matrix $\mathbf{R}$, using Kronecker algebra, for a Markov model with K submodels. Our algorithm builds and solves K Markov chains, each corresponding to a different aggregation of the exact process, guided by the structure of the decision diagram, and iterates on their solution until their entries are stable. We prove that exact results are obtained if the overall model has a product-form solution. Advantages of our method include good accuracy, low memory requirements, fast execution times, and a high degree of automation, since the only additional information required to apply it is a partition of the model into the K submodels. As far as we know, this is the first time an approximation algorithm has been proposed where knowledge of the exact state space is explicitly used.

1. INTRODUCTION

Fixed-point approximation algorithms for the solution of large Markov models have been widely used [7; 12; 15; 16; 17; 22; 25; 26; 27]. The idea is to “decompose” the original model into submodels and then somehow use the results obtained from the solution of the submodels to update their rate parameters, until the iteration converges. While this idea can work quite well in practice and drastically reduce the solution time with respect to that of the overall exact model (which might not even be feasible), there are several potential pitfalls: the iteration might fail to converge, or it might converge to different values depending on the initial guess, or it might converge to a value very far from the

*This work was partially performed while A.S. Miner and G. Ciardo were visiting the Università di Torino. A.S. Miner’s work was supported by fellowships from the NASA Graduate Student Research Program (NGT-1-52195) and the Virginia Space Grant Consortium. G. Ciardo was supported by a NASA grant (NAG-1-2168), by a Semester Research Assignment from the College of William and Mary, and by a travel grant from Italy’s Consiglio Nazionale delle Ricerche.

exact result. Proofs of convergence are usually difficult to obtain [18]. A further problem not often mentioned is that, when submodels are combined, “spurious” states might be introduced, which do not exist in the exact model. This can be a problem, as, for example, it might suggest that certain events can occur while they have probability zero, or it might negatively affect the quality of the results.

Recent advances in decision diagram techniques [1; 2; 19; 20; 23] and the use of Kronecker algebra [3; 4; 5; 11; 13; 14; 21; 24] can be used to store the state space and the transition rate matrix of a huge Markov model very efficiently, but the size of the probability vector still precludes us from obtaining an exact solution for such a process. In this work, we present a new fixed-point iteration algorithm that uses this “exact” representation of the model to compute an approximation of the probability vector and of the desired measures. As far as we know, this is the first time an approximation algorithm has been proposed where complete knowledge of the state space is explicitly exploited.

2. COMPOSITIONAL DESCRIPTION

Assume that the state space $\mathcal{S}$ of the system under study is a subset of $\hat{\mathcal{S}} = \mathcal{S}_K \times \dots \times \mathcal{S}_1$, where $\mathcal{S}_k = \{0, 1, \dots, n_k - 1\}$, $n_k \geq 2$, is the “local state space” at level k . Hence, a system state is a tuple of the form $\mathbf{i} = (i_K, \dots, i_1)$. For example, given a queueing network or a stochastic Petri net model of a system, we can partition its queues, or places, into K subsets, thus identifying K submodels (in our figures, this is indicated by grey oval shapes surrounding queues, or places). The simplest case occurs when each queue, or place, is in a different submodel; the state is then simply the vector of queue occupancies, or the marking of the net (number of tokens in each place). However, such a fine partition of the model into submodels is not always advisable from the point of view of solution efficiency, or even possible, since we will see in Sect. 2.1 that the dynamic description of the submodels must satisfy certain compositional restrictions.

The model has a finite set of events $\mathcal{E}$. An event $e \in \mathcal{E}$ can move the system from the current state $\mathbf{i}$ to any of the states in set $\mathcal{X}^e(\mathbf{i})$. If $\mathcal{X}^e(\mathbf{i}) = \emptyset$, we say that e is disabled in $\mathbf{i}$, otherwise we say it is enabled. The (global) state space set $\mathcal{S}$ is formally defined as the smallest subset of $\hat{\mathcal{S}}$ that contains the initial state, part of the given system’s description, and is closed under the one-step reachability relation; that is, if $\mathbf{i} \in \mathcal{S}$, then $\mathcal{X}^e(\mathbf{i}) \subseteq \mathcal{S}$, for any event e .

We also need to specify the state-to-state transition rates, that is, the transition rate matrix $\mathbf{R}$. This can be done through two functions: $\lambda^e(\mathbf{i})$, giving the rate of the exponentially distributed duration for the occurrence of event e when the state is $\mathbf{i}$, and $\text{Prob}^e(\mathbf{i}, \mathbf{j})$, giving the probability that, if event e occurs in state $\mathbf{i}$, the state reached is $\mathbf{j}$:

$$\mathbf{R}[\mathbf{i}, \mathbf{j}] = \sum_{e \in \mathcal{E}} \mathbf{R}^e[\mathbf{i}, \mathbf{j}] = \sum_{e \in \mathcal{E}} \lambda^e(\mathbf{i}) \cdot \text{Prob}^e(\mathbf{i}, \mathbf{j}),$$

where $\mathbf{R}^e \in \mathbb{R}^{|\mathcal{S} \times \mathcal{S}|}$ is the contribution due to event e .

2.1 Using Kronecker operators to store $\mathbf{R}$

We can express $\mathbf{R}^e$ with Kronecker operators as proposed by Plateau [14; 21] when $\mathcal{X}^e$, λ^e , and Prob^e can be expressed compositionally:

$$\begin{aligned} \mathcal{X}^e(\mathbf{i}) &= \mathcal{X}_K^e(i_K) \times \cdots \times \mathcal{X}_1^e(i_1) \\ \lambda^e(\mathbf{i}) &= \lambda_K^e[i_K] \cdots \lambda_1^e[i_1] \\ \text{Prob}^e(\mathbf{i}, \mathbf{j}) &= \text{Prob}_K^e(i_K, j_K) \cdots \text{Prob}_1^e(i_1, j_1), \end{aligned}$$

where $\mathcal{X}_k^e$ and Prob_k^e are functions applied to submodel k in isolation, and λ_k^e is a vector of size $|\mathcal{S}_k|$. This compositional requirement restricts our choice of model decomposition somewhat. For instance, if the rate of an event depends upon the sum of state variables i and j , then our decomposition must assign i and j to the same submodel. When the compositional requirement is satisfied, we can define [4] the matrix $\hat{\mathbf{R}}^e \in \mathbb{R}^{|\hat{\mathcal{S}} \times \hat{\mathcal{S}}|}$ as

$$\hat{\mathbf{R}}^e = \left(\bigotimes_{k=1}^K \mathbf{W}_k^e \right), \quad (1)$$

where $\mathbf{W}_k^e \in \mathbb{R}^{|\mathcal{S}_k \times \mathcal{S}_k|}$ gives the effect of e on submodel k :

$$\mathbf{W}_k^e[i_k, j_k] = \lambda_k^e[i_k] \cdot \text{Prob}_k^e(i_k, j_k).$$

Note that elements in row i_k of $\mathbf{W}_k^e$ sum to $\lambda_k^e[i_k]$. Matrix $\mathbf{R}^e$ is then the submatrix of $\hat{\mathbf{R}}^e$ corresponding to the reachable states: $\mathbf{R}^e = \hat{\mathbf{R}}^e[\mathcal{S}, \mathcal{S}]$. That is, the matrix $\hat{\mathbf{R}}^e$ may contain extra rows and columns due to unreachable states; this occurs when $\mathcal{S}$ is a strict subset of $\hat{\mathcal{S}}$. Thus, instead of storing $\mathbf{R}$ explicitly, we compute the elements of $\mathbf{R}^e$ as needed (by selecting the reachable portion of $\hat{\mathbf{R}}^e$, computed from the much smaller $\mathbf{W}$ matrices using Eq. 1) and sum over all events $e \in \mathcal{E}$. The infinitesimal generator $\mathbf{Q}$ of the underlying continuous-time Markov chain (CTMC) is the reachable portion of $\hat{\mathbf{Q}}$:

$$\mathbf{Q} = \hat{\mathbf{Q}}[\mathcal{S}, \mathcal{S}], \quad \hat{\mathbf{Q}} = \left(\sum_{e \in \mathcal{E}} \bigotimes_{k=1}^K \mathbf{W}_k^e - \bigotimes_{k=1}^K \text{diag}(\lambda_k^e) \right),$$

where “diag” indicates a square matrix having the argument, a vector, on its main diagonal and zeros elsewhere.

We say that e is independent of level k if $\mathcal{X}_k^e(i_k) = \{i_k\}$ for all local states $i_k \in \mathcal{S}_k$ and $\mathbf{W}_k^e$ is the identity matrix. Otherwise, we say that e depends on level k . If e depends only on a single level k , we say that it is a local event for level k ; otherwise it is a synchronizing event. Let $\text{First}(e)$ and $\text{Last}(e)$ be the maximum and minimum levels on which e depends, noting that e might or might not depend on levels strictly between $\text{First}(e)$ and $\text{Last}(e)$.

2.2 Using decision diagrams to store $\mathcal{S}$

Decision diagrams have recently been used to build and store $\mathcal{S} \subseteq \hat{\mathcal{S}}$ extremely efficiently for most systems of interest. We

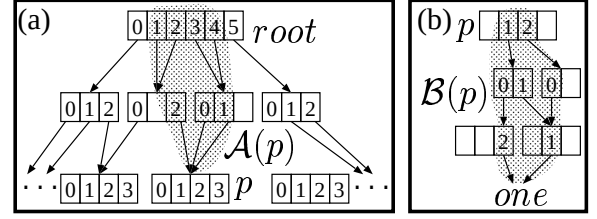


Figure 1: Sets encoded by decision diagrams.

now briefly describe the relevant definitions and terminology, more detailed information and further references can be found in [8; 19]; a few illustrative examples are given at the end of this section. In our particular application, a decision diagram is a directed acyclic graph whose nodes are organized in levels ranging from K , at the top, to 1, at the bottom, plus the special level 0, which contains only a special node *one* corresponding to the boolean value 1 (in practice, there is no need to store this level explicitly). There is a single root node, *root*, at level K without incoming arcs. Any other node is at a level below K and has one or more incoming arcs, from the level immediately above. A node p at level k , $K \geq k \geq 1$, is then essentially an array of n_k pointers, $p[0], \dots, p[n_k - 1]$ to nodes at level $k - 1$ or to null, corresponding to the boolean value 0 at level 0. We use a special version of “reduced ordered decision diagrams” (written “decision diagrams” from here on) where no node can be a “replica” of another node at the same level (that is, have exactly the same p array), but “dummy” nodes are allowed (nodes for which all the entries in p have the same non-null value). Most definitions forbid dummy nodes, but we allow them since we require arcs to go between adjacent levels only. A fundamental property of decision diagrams (reduced according to either the classical definition or the one with dummy nodes we use) is that they are a canonical representation of sets.

The concept of reachable substate is fundamental to our discussion. Let $\mathcal{L}[k]$ be the set of nodes at level k . Given a node $p \in \mathcal{L}[k]$, we can then recursively define the set $\mathcal{B}(p)$ of substates encoded by (or “below”) p as

$$\mathcal{B}(p) = \begin{cases} \bigcup_{p[i] \neq \text{null}} \{i\} \times \mathcal{B}(p[i]) & \text{if } k > 1 \\ \{i \in \mathcal{S}_1 : p[i] \neq \text{null}\} & \text{if } k = 1 \end{cases},$$

where $\mathcal{B}(\text{null}) = \emptyset$ by definition. We can then store our reachability set $\mathcal{S}$ with a K -level decision diagram rooted at node *root*: $\mathcal{S} = \mathcal{B}(\text{root})$. Given a decision diagram representation of $\mathcal{S}$, a substate $\alpha = (i_K, \dots, i_k)$ is said to be reachable if there is a state in $\mathcal{S}$ beginning with α . Such substate uniquely identifies a node at level k :

$$\text{node}(\alpha) = \begin{cases} \text{root} & \text{if } |\alpha| = 0 \\ \text{node}(i_K, \dots, i_{k+1})[i_k] & \text{otherwise} \end{cases}.$$

We can then define the set $\mathcal{A}(p)$ of substates leading to node p (or “above” p) as $\mathcal{A}(p) = \{\alpha : \text{node}(\alpha) = p\}$. In particular, $\mathcal{A}(\text{one}) = \mathcal{S}$.

We illustrate the use of decision diagrams for storing a state space with a few simple examples. Fig. 1 provides examples of sets $\mathcal{A}(p)$ and $\mathcal{B}(p)$. The example in Fig. 1(a) shows $\mathcal{A}(p) = \{12, 22, 30, 31, 40, 41\}$, and Fig. 1(b) shows $\mathcal{B}(p) = \{102, 111, 201\}$. Examples of simple models and the decision

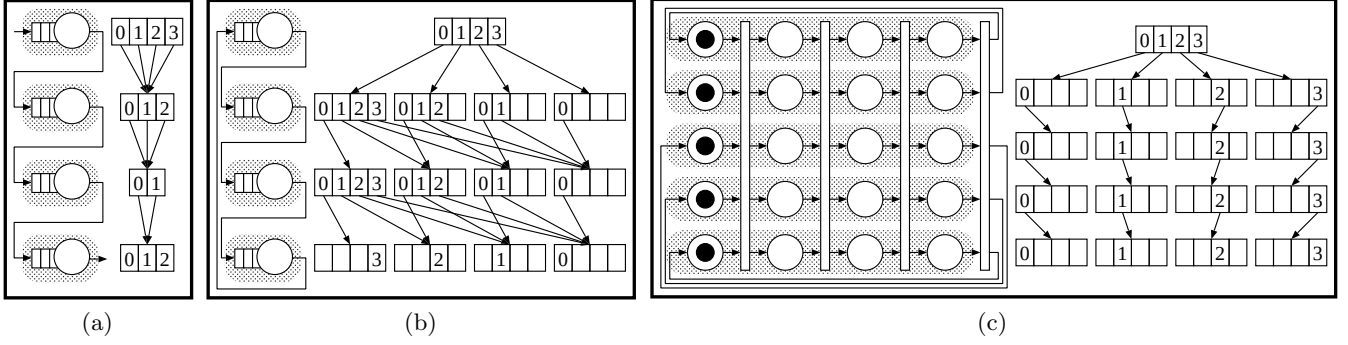


Figure 2: Models and the decision diagrams encoding their state spaces.

diagram representations for their state spaces are shown in Fig. 2. In the figure, we show the local state index in the array positions having a non-null pointer, and we omit pointers at level 1, since they go to node *one*. Consider first an open queueing network with $Q = 4$ tandem queues and bounds 3, 2, 1, and 2, respectively, on the size of their buffer. The model and the resulting decision diagram, assuming each queue is assigned a different level, is shown in Fig. 2(a). For this model, $\hat{\mathcal{S}} = \mathcal{S}$, hence the decision diagram has a single node per level. A more complex example is that of a single-class closed queueing network with $Q = 4$ queues and $N = 3$ customers, shown in Fig. 2(b). The local state spaces at each level are all the same, $\mathcal{S}_k = \{0, 1, 2, 3\}$, and some combinations of local states cannot occur.

Decision diagrams do have pathological cases with space requirements $O(K \cdot |\mathcal{S}|)$, the same as if we were storing $\mathcal{S}$ using an ordinary search tree with one state per node, stored as a full array of K substates. Consider for example the fully synchronized Petri net of Fig. 2(c). Its state space is $\mathcal{S} = \{[0, 0, 0, 0, 0], [1, 1, 1, 1, 1], [2, 2, 2, 2, 2], [3, 3, 3, 3, 3]\}$, but the bad choice of partition (or, perhaps, to partition at all) creates $|\mathcal{S}| = 4$ chains of nodes, each $K - 1$ tall, plus a single root node. This model is just an extreme case of a problem already visible in the closed queueing network of Fig. 2(b), where each node at level 1 contains exactly one non-null pointer. This means that any path to such a node, i.e., any substate $(i_4, i_3, i_2) \in \mathcal{A}(p)$, uniquely determines a local state i_1 . Clearly, this is a waste that can be eliminated by merging levels 1 and 2, that is, merging the corresponding submodels. For the closed queueing network example, this reflects the fact that the number of customers N in the network is fixed: if $L = i_4 + i_3 + i_2$ customers are in the first 3 queues, $i_1 = N - L$ customers must be in the last queue. For the fully synchronized Petri net, the idea can be applied at each level except the first one, resulting in a single submodel corresponding to the entire model.

3. OUR APPROXIMATE AGGREGATION

We now present our main idea, after briefly recalling the concept of (exact) aggregation for a CTMC.

3.1 Exact aggregation

Given a CTMC with state space $\mathcal{S}$, transition rate matrix $\mathbf{R}$, and stationary probability vector $\boldsymbol{\pi}$, partition $\mathcal{S}$ into C disjoint classes $\mathcal{C}_1, \dots, \mathcal{C}_C$ and define an aggregated CTMC with state space $\{1, \dots, C\}$ and transition rate matrix $\mathbf{R}_{aggr}$

given by

$$\mathbf{R}_{aggr}[i, j] = \sum_{\mathbf{i} \in \mathcal{C}_i} \boldsymbol{\pi}[\mathbf{i} | \mathcal{C}_i] \cdot \sum_{\mathbf{j} \in \mathcal{C}_j} \mathbf{R}[\mathbf{i}, \mathbf{j}], \quad (2)$$

where $\boldsymbol{\pi}[\mathbf{i} | \mathcal{C}_i]$ is the conditional probability that the original CTMC is in state $\mathbf{i}$ given that it is in $\mathcal{C}_i$:

$$\boldsymbol{\pi}[\mathbf{i} | \mathcal{C}_i] = \boldsymbol{\pi}[\mathbf{i}] \cdot \left(\sum_{\mathbf{j} \in \mathcal{C}_i} \boldsymbol{\pi}[\mathbf{j}] \right)^{-1}. \quad (3)$$

Thus, $\mathbf{R}_{aggr}[i, j]$ represents the stationary rate at which the original CTMC goes from $\mathcal{C}_i$ to $\mathcal{C}_j$ given that it is in $\mathcal{C}_i$.

If the original CTMC is ergodic, so is the aggregated CTMC, and its stationary probability vector $\boldsymbol{\pi}_{aggr}$ satisfies $\boldsymbol{\pi}_{aggr}[i] = \sum_{\mathbf{i} \in \mathcal{C}_i} \boldsymbol{\pi}[\mathbf{i}]$. While exact aggregation allows us in principle to greatly reduce the size of the state space to be considered, its use is rarely practical, since the computation of the aggregate rates can be correctly performed in general only if the stationary probability vector $\boldsymbol{\pi}$ of the original CTMC is known. In practice, we often resort to approximate aggregation, where an estimate of the conditional probabilities $\boldsymbol{\pi}[\mathbf{i} | \mathcal{C}_i]$ is used instead of the exact expression of Eq. 3.

3.2 Partition based on decision diagrams

We now apply aggregation based on a partition of $\mathcal{S}$ guided by the decision diagram structure discussed in Sect. 2.2. We will build K aggregated CTMCs, one per level of the decision diagram. The states in the state space $\mathcal{M}_k$ of the level- k CTMC are pairs of the form (p, i_k) where p is a node at level k and i_k is a reachable local state for that node:

$$\mathcal{M}_k = \{(p, i_k) : p \in \mathcal{L}[k] \wedge p[i_k] \neq \text{null}\}.$$

$\mathcal{M}_k$ induces the partition $\mathcal{S} = \bigcup_{(p, i_k) \in \mathcal{M}_k} \mathcal{C}(p, i_k)$, where

$$\mathcal{C}(p, i_k) = \mathcal{A}(p) \times \{i_k\} \times \mathcal{B}(p[i_k]).$$

Intuitively, class $\mathcal{C}(p, i_k)$ contains the states with i_k as the local state for submodel k whose path in the decision diagram from node *root* to node *one* passes through node p . For an exact aggregation, we derive the rate from (p, i_k) to (q, j_k) in the level- k CTMC due to event e by combining

Eq. 1 and Eq. 2:

$$\begin{aligned}
\mathbf{R}_k^e[(p, i_k), (q, j_k)] &= \sum_{i \in \mathcal{C}(p, i_k)} \pi[i | \mathcal{C}(p, i_k)] \cdot \sum_{j \in \mathcal{C}(q, j_k)} \mathbf{R}^e[i, j] \\
&\quad \left(\text{let } \begin{array}{l} \alpha = i_K, \dots, i_{k+1}, \beta = i_{k-1}, \dots, i_1, \\ \alpha' = j_K, \dots, j_{k+1}, \beta' = j_{k-1}, \dots, j_1 \end{array} \right) \\
&= \sum_{\alpha \in \mathcal{A}(p), \beta \in \mathcal{B}(p[i_k])} \pi[(\alpha, i_k, \beta) | \mathcal{C}(p, i_k)] \cdot \sum_{\alpha' \in \mathcal{A}(q), \beta' \in \mathcal{B}(q[j_k])} \mathbf{W}_K^e[i_K, j_K] \cdots \mathbf{W}_1^e[i_1, j_1] \\
&= \sum_{\alpha \in \mathcal{A}(p)} \Pr\{\alpha | p, i_k\} \cdot \sum_{\alpha' \in \mathcal{A}(q)} \mathbf{W}_K^e[i_K, j_K] \cdots \mathbf{W}_k^e[i_k, j_k] \cdot \\
&\quad \sum_{\beta \in \mathcal{B}(p[i_k])} \Pr\{\beta | \alpha, i_k\} \cdot \lambda_{k-1}^e[i_{k-1}] \cdots \lambda_1^e[i_1]. \tag{4}
\end{aligned}$$

If the original CTMC is ergodic, then the level- k CTMC is ergodic and has stationary probability vector π_k .

3.3 A simple case of exact aggregation

We first discuss a special case where exact aggregation naturally arises, as preparation to our approximation approach. Consider a decision diagram where replicas are not only allowed, but extensively required by the fact that we forbid two arcs from pointing to the same node. We can visualize such an “unreduced decision diagram” as a tree where the nodes at level k have anywhere between one and n_k pointers to nodes at level $k-1$. This is the multilevel data structure introduced in [10] to store $\mathcal{S}$. For example, the open queueing network of Fig. 2(a) would have a decision diagram with 1, $n_4 = 4$, $n_4 \cdot n_3 = 12$, and $n_4 \cdot n_3 \cdot n_2 = 24$ nodes at level 4, 3, 2, and 1, respectively, while the closed queueing network of Fig. 2(b) would have a decision diagram with 1, $\binom{3+1}{1} = 4$, $\binom{3+2}{2} = 10$, and $\binom{3+3}{3} = 20$ nodes at level 4, 3, 2, and 1, respectively; note that the Petri net of Fig. 2(c) is already in the required form.

If the decision diagram is a tree, there is exactly one path to each node p ($|\mathcal{A}(p)| = 1$). Thus the level-1 CTMC is not really aggregated, in the sense that each class in the partition at level 1 contains one state of the original CTMC, $|\mathcal{M}_1| = |\mathcal{S}|$. Then,

$$\Pr\{\alpha | p, i_k\} = 1 \quad \text{and} \quad \Pr\{\beta | \alpha, i_k\} = \Pr\{\beta | p, i_k\}$$

and we can rewrite Eq. 4 for $\mathbf{R}_k^e[(p, i_k), (q, j_k)]$ as

$$\mathbf{R}_k^e[(p, i_k), (q, j_k)] = \mathbf{A}_k^e[p, q] \cdot \mathbf{W}_k^e[i_k, j_k] \cdot \mathbf{b}_k^e[p[i_k]]$$

where

$$\mathbf{A}_k^e[p, q] = \mathbf{W}_K^e[i_K, j_K] \cdots \mathbf{W}_{k+1}^e[i_{k+1}, j_{k+1}]$$

expresses the contribution to the rate “from above” and

$$\mathbf{b}_k^e[p[i_k]] = \sum_{\beta \in \mathcal{B}(p[i_k])} \Pr\{\beta | p, i_k\} \cdot \lambda_{k-1}^e[i_{k-1}] \cdots \lambda_1^e[i_1]$$

expresses the contribution “from below”. Defining the terminal cases $\mathbf{A}_K^e[\text{root}, \text{root}] = 1$ and $\mathbf{b}_k^e[\text{one}] = 1$, we can express $\mathbf{A}$ and $\mathbf{b}$ recursively: if $p = p'[i_{k+1}]$ and $q = q'[j_{k+1}]$,

$$\begin{aligned}
\mathbf{A}_k^e[p, q] &= \mathbf{W}_{k+1}^e[i_{k+1}, j_{k+1}] \cdot \mathbf{A}^e[p', q'] \\
\mathbf{b}_k^e[p] &= \sum_{i_k: p[i_k] \neq \text{null}} \Pr\{i_k | p\} \cdot \lambda_k^e[i_k] \cdot \mathbf{b}_k^e[p[i_k]].
\end{aligned}$$

At level 1, the “aggregated” CTMC and the exact CTMC coincide, thus $\pi_1 = \pi$ and the recursive computation of $\mathbf{b}$ results in the correct conditional probabilities $\Pr\{i_k | p\}$ being used at every level. In other words, every level- k CTMC

is an exact aggregation of the original CTMC. Thus, this is an illustrative case only, since there is no point in defining aggregated CTMCs once the exact CTMC has been solved.

3.4 Our approximation

Armed with the intuition from the previous section, we are now ready to deal with the case where the decision diagram is indeed reduced, and multiple paths could reach node p , that is, $|\mathcal{A}(p)| \geq 1$. Before deriving the expression for the aggregated CTMCs, we introduce the only source of our approximation. Our goal is to obtain stationary probability vectors for the CTMCs at each level that can be used to approximate the entire stationary probability vector for the original CTMC. To do so, we assume that

$$\forall \alpha \in \mathcal{A}(p), \quad \Pr\{i_k | \alpha\} = \Pr\{i_k | p\}, \tag{5}$$

or, equivalently,

$$\forall \alpha, \alpha' \in \mathcal{A}(p), \quad \Pr\{i_k | \alpha\} = \Pr\{i_k | \alpha'\},$$

which implies that p captures all the information about the path α , from *root* to p , required to correctly compute the rate of an event e .

Of course, this is a strong assumption, and the quality of the approximation will ultimately depend on how closely it holds. However, this assumption has an excellent structural justification: the set of substates (i_k, β) that can complete a given α to form a reachable state (α, i_k, β) is exactly the same for any $\alpha \in \mathcal{A}(p)$. This implies that, given $\alpha, \alpha' \in \mathcal{A}(p)$, $\Pr\{i_k | \alpha\} = 0 \Leftrightarrow \Pr\{i_k | \alpha'\} = 0$, and is indeed one of the strengths of our approximation algorithm: we can, at the very least, exclude from consideration “spurious” states in $\hat{\mathcal{S}} \setminus \mathcal{S}$, since we have an exact encoding of the state space $\mathcal{S}$. What is not true in general, however, is that $\Pr\{i_k | \alpha\} = \Pr\{i_k | \alpha'\}$ when they are not zero.

Using our assumption, the stationary probability of state $(i_K, \dots, i_1)$ of the original CTMC is then

$$\begin{aligned}
\Pr\{i_K, \dots, i_1\} &= \Pr\{i_1 | i_K, \dots, i_2\} \cdot \Pr\{i_K, \dots, i_2\} = \\
&= \Pr\{i_1 | p_1\} \cdot \Pr\{i_K, \dots, i_2\} = \prod_{k=1}^K \Pr\{i_k | p_k\} = \prod_{k=1}^K \frac{\pi_k[(p_k, i_k)]}{\Pr\{p_k\}},
\end{aligned}$$

where $p_k = \text{node}(i_K, \dots, i_{k+1})$, $\pi_k[(p_k, i_k)]$ is the stationary probability of state (p_k, i_k) in the level- k CTMC, and, of course, $\Pr\{p_K\} = \Pr\{\text{root}\} = 1$.

Using Eq. 5, $\Pr\{\alpha | p, i_k\}$ can be written as

$$\frac{\Pr\{\alpha, p, i_k\}}{\Pr\{p, i_k\}} = \frac{\Pr\{p, i_k | \alpha\} \Pr\{\alpha\}}{\Pr\{i_k | p\} \Pr\{p\}} = \frac{\Pr\{i_k | \alpha\} \Pr\{\alpha\}}{\Pr\{i_k | p\} \Pr\{p\}} = \frac{\Pr\{\alpha\}}{\Pr\{p\}}.$$

Analogously, we can show that $\Pr\{\beta | \alpha, i_k\} = \Pr\{\beta | p, i_k\}$, since the dependency of β on α can be eliminated recursively: letting $\beta = (i_{k-1}, \dots, i_1) = (i_{k-1}, \gamma)$, we have

$$\begin{aligned}
\Pr\{\beta | \alpha, i_k\} &= \Pr\{i_{k-1}, \gamma | \alpha, i_k\} \\
&= \Pr\{i_{k-1} | \alpha, i_k\} \cdot \Pr\{\gamma | \alpha, i_k, i_{k-1}\} \\
&= \Pr\{i_{k-1} | p[i_k]\} \cdot \Pr\{\gamma | \alpha, i_k, i_{k-1}\} \\
&= \Pr\{i_{k-1} | p[i_k]\} \cdots \Pr\{i_1 | p[i_k][i_{k-1}] \cdots [i_2]\}.
\end{aligned}$$

Substituting into Eq. 4 we then express $\mathbf{R}_k^e[(p, i_k), (q, j_k)]$ as

$$\begin{aligned} & \left(\sum_{\alpha \in \mathcal{A}(p)} \frac{Pr\{\alpha\}}{Pr\{p\}} \cdot \sum_{\alpha' \in \mathcal{A}(q)} \mathbf{W}_K^e[i_K, j_K] \cdots \mathbf{W}_k^e[i_{k+1}, j_{k+1}] \right) \cdot \\ & \mathbf{W}_k^e[i_k, j_k] \cdot \left(\sum_{\beta \in \mathcal{B}(p[i_k])} Pr\{\beta|p, i_k\} \cdot \lambda_{k-1}^e[i_{k-1}] \cdots \lambda_1^e[i_1] \right) \\ & = \mathbf{A}_k^e[p, q] \cdot \mathbf{W}_k^e[i_k, j_k] \cdot \mathbf{b}_{k-1}^e[p[i_k]] \end{aligned} \quad (6)$$

where $\mathbf{A}_k^e[p, q]$ can be written as the recurrence

$$\begin{aligned} \mathbf{A}_k^e[p, q] &= \sum_{(i_K, \dots, i_{k+1}) \in \mathcal{A}(p)} \frac{Pr\{i_K, \dots, i_{k+1}\}}{Pr\{p\}} \\ & \cdot \sum_{(j_K, \dots, j_{k+1}) \in \mathcal{A}(q)} \mathbf{W}_K^e[i_K, j_K] \cdots \mathbf{W}_{k+1}^e[i_{k+1}, j_{k+1}] \\ &= \sum_{p', i_{k+1}: p'[i_{k+1}] = p} \frac{Pr\{p', i_{k+1}\}}{Pr\{p\}} \\ & \cdot \sum_{q', j_{k+1}: q'[j_{k+1}] = q} \mathbf{A}_{k+1}^e[p', q'] \cdot \mathbf{W}_{k+1}^e[i_{k+1}, j_{k+1}] \end{aligned}$$

which can be proved by letting $\alpha = (i_K, \dots, i_{k+1}) = (\gamma, i_{k+1})$ and $p' = \text{node}(\gamma)$, and observing that

$$\begin{aligned} \frac{Pr\{\gamma, i_{k+1}\}}{Pr\{p\}} &= \frac{Pr\{i_{k+1}|\gamma\} \cdot Pr\{\gamma\}}{Pr\{p\}} = \frac{Pr\{i_{k+1}|p'\} \cdot Pr\{\gamma\}}{Pr\{p\}} \\ &= \frac{Pr\{i_{k+1}, p'\}}{Pr\{p\}} \cdot \frac{Pr\{\gamma\}}{Pr\{p'\}}. \end{aligned}$$

Analogously, $\mathbf{b}$ can be written as the recurrence

$$\begin{aligned} \mathbf{b}_k^e[p] &= \sum_{(i_k, \dots, i_1) \in \mathcal{B}(p)} Pr\{i_k, \dots, i_1|p\} \cdot \lambda_k^e[i_k] \cdots \lambda_1^e[i_1] \\ &= \sum_{i_k: p[i_k] \neq \text{null}} \frac{Pr\{p, i_k\}}{Pr\{p\}} \cdot \lambda_k^e[i_k] \cdot \mathbf{b}_k^e[p[i_k]]. \end{aligned}$$

The terminal cases for the recursions are as for the exact aggregation case, $\mathbf{A}_K^e[\text{root}, \text{root}] = 1$ and $\mathbf{b}_0^e[\text{one}] = 1$.

Thus, we need to compute probabilities of the type $Pr\{p\}$ and $Pr\{p, i_k\}$, for each node p at each level k and each local state i_k such that $p[i_k] \neq \text{null}$. In our algorithm, $Pr\{p, i_k\}$ is simply $\pi_k[(p, i_k)]$, the entry of the (current guess of the) level- k stationary probability vector for state (p, i_k) . To compute $Pr\{p\}$, we use the expression

$$Pr\{p\} = \sum_{i_k: p[i_k] \neq \text{null}} \pi_k[(p, i_k)]$$

and perform the sum for each node p after computing the level- k stationary probability vector. We could also use

$$Pr\{p\} = \sum_{\alpha \in \mathcal{A}(p)} Pr\{\alpha\}$$

but the former is preferable because it is a simple sum, while the latter requires us to recursively compute $Pr\{\alpha\}$ (the number of paths $\alpha \in \mathcal{A}(p)$ can grow exponentially as we descend the decision diagram, but the recursive computation makes use of previous results, thus its cost is proportional to the number of *nodes* above p , not to the number of *paths* leading to p). It is comforting to know that, upon convergence the values obtained with either method coincide.

3.5 Exploiting event locality

Considering that both the enabling and the effect of an event e are restricted to the levels between $\text{First}(e)$ and $\text{Last}(e)$, we can further simplify the above recurrence expressions. For $k > \text{First}(e)$, $\mathbf{W}_k^e$ is the identity, hence for $k \geq \text{First}(e)$,

$$\mathbf{A}_k^e[p, q] = \begin{cases} \sum_{(i_K, \dots, i_{k+1}) \in \mathcal{A}(p)} \frac{Pr\{i_K, \dots, i_{k+1}\}}{Pr\{p\}} = 1 & \text{if } p = q \\ 0 & \text{otherwise} \end{cases},$$

i.e., e can be ignored for $k > \text{First}(e)$, and e only changes the local state component of the state (p, i_k) at level $k = \text{First}(e)$, but not the node component: $\mathbf{R}_k^e[(p, i_k), (p, j_k)] = \mathbf{W}_k^e[i_k, j_k] \cdot \mathbf{b}_{k-1}^e[p[i_k]]$ and $\mathbf{R}_k^e[(p, i_k), (q, j_k)] = 0$ for $p \neq q$.

Analogously, since $\mathbf{W}_k^e$ is also an identity for $k < \text{Last}(e)$, which implies that $\lambda_k^e[i_k] = 1$ for any $i_k \in \mathcal{S}_k$, we have

$$\forall k < \text{Last}(e), \mathbf{b}_k^e[p] = \sum_{(i_k, \dots, i_1) \in \mathcal{B}(p)} Pr\{i_k, \dots, i_1|p\} = 1,$$

i.e., for $k = \text{Last}(e)$, $\mathbf{R}_k^e[(p, i_k), (q, j_k)] = \mathbf{A}_k^e[p, q] \cdot \mathbf{W}_k^e[i_k, j_k]$ and, for $k < \text{Last}(e)$, e can change the node, but not the local state, component of state (p, i_k) : $\mathbf{R}_k^e[(p, i_k), (q, i_k)] = \mathbf{A}_k^e[p, q]$ and $\mathbf{R}_k^e[(p, i_k), (q, j_k)] = 0$ for $i_k \neq j_k$.

In particular, if e is a local event for level k , we obtain that the only transitions caused by e are

$$\mathbf{R}_k^e[(p, i_k), (p, j_k)] = \mathbf{W}_k^e[i_k, j_k] \quad \text{for } j_k \in \mathcal{X}_k^e(i_k).$$

As these rates are exact (they involve no approximation), a simple rule of thumb in the hope of achieving a good approximate aggregation is to partition the model in a way that minimizes the number of synchronizing events. Of course, this is just a heuristic; we will see that, for models exhibiting a product-form solution, our algorithm computes exact results even if *every* event is synchronizing.

We now summarize, giving an intuitive explanation for the effect of locality on the transition rates in $\mathbf{R}_k^e$. Consider first the effect of a local event e for level k . Since e is independent of all other levels, we know that

$$\mathcal{X}^e((\alpha, i_k, \beta)) = \alpha \times \mathcal{X}_k^e(i_k) \times \beta$$

and the rate of e is $\mathbf{W}_k^e[i_k, j_k]$ for each $j_k \in \mathcal{X}_k^e(i_k)$. Thus, for each state in $\mathcal{C}(p, i_k)$, we know that event e is enabled, and leads to states in $\mathcal{C}(p, j_k)$ for $j_k \in \mathcal{X}_k^e(i_k)$. In the level- k CTMC, this is reflected by transitions from state (p, i_k) to state (p, j_k) with rate $\mathbf{W}_k^e[i_k, j_k]$ for all $(p, i_k) \in \mathcal{M}_k$. It is also possible for e to affect states in the level k' CTMC, for $k' < k$. Considering the case where $k' = k - 1$, we see that e causes a transition from $(p[i_k], i_{k'})$ to $(p[j_k], i_{k'})$ in the level- k' CTMC for all local states $i_{k'}$ such that $(p[i_k], i_{k'}) \in \mathcal{M}_{k'}$. The total rate of this transition is given by $\mathbf{A}_k^e[p[i_k], p[j_k]]$, which is the sum over all nodes p' such that $p'[i_k] = p[i_k]$ and $p'[j_k] = p[j_k]$ of the probability of being in state (p', i_k) given that we are in node $p[i_k]$, multiplied by the rate of the transition from i_k to j_k at level k . This transition will continue to affect later levels until all the arcs corresponding to the transition are self-arcs. In particular, if $p[i_k] = p[j_k]$, then the corresponding arcs at the next level will all be self-arcs, and e will not affect any other levels.

For a synchronizing event e , there are four distinct cases:

- $k > \text{First}(e)$. In this trivial case, e does not affect level k , nor does it affect any levels above it. Any transition due to e results in a self-arc in the level- k CTMC, so we can simply ignore the effects of e at this level.
- $k = \text{First}(e)$. This case is analogous to that of local events. We know that $\mathcal{X}^e((\alpha, i_k, \beta)) = \alpha \times \mathcal{X}_k^e(i_k) \times \beta'$ for some β' . As the α portion of the state does not change, the corresponding transition due to e in the level- k CTMC will be from state (p, i_k) to state (p, j_k) for $j_k \in \mathcal{X}_k^e(i_k)$. Unlike a local event, though, e might be enabled in all, some, or

none of the states in $\mathcal{C}(p, i_k)$. Thanks to our exact encoding of the state space $\mathcal{S}$, we can detect when e is not enabled in any state in $\mathcal{C}(p, i_k)$; if this is the case, we do not have a corresponding arc in the level- k CTMC. On the other hand, if e is enabled in some or all of the states in $\mathcal{C}(p, i_k)$, we place a corresponding arc between state (p, i_k) and all the $(p, j_k) \in \mathcal{M}_k$, for $j_k \in \mathcal{X}_k^e(i_k)$. The rate of each arc is given by $\mathbf{W}_k^e[i_k, j_k] \cdot \mathbf{b}_{k-1}^e[p[i_k]]$, which is the product of the contribution at level k and the one “from below”.

- *First*(e) $> k \geq$ *Last*(e). This is the most general case, as we have $\mathcal{X}^e((\alpha, i_k, \beta)) = \alpha' \times \mathcal{X}_k^e(i_k) \times \beta'$ for some α', β' . If e is enabled in at least one state in $\mathcal{C}(p, i_k)$, the level- k CTMC has arcs from state (p, i_k) to states $(p', j_k) \in \mathcal{M}_k$, for $j_k \in \mathcal{X}_k^e(i_k)$, where $p' = \text{node}(\alpha')$. The rate of each arc is given by $\mathbf{A}_k^e[p, p'] \cdot \mathbf{W}_k^e[i_k, j_k] \cdot \mathbf{b}_{k-1}^e[p[i_k]]$, which is the product of the contribution from above, from level k , and from below. If $k = \text{Last}(e)$, $\mathbf{b}_{k-1}^e[p[i_k]] = 1$, since e does not affect any level below.

- *Last*(e) $> k$. This is similar to the case when an event local to level k affects levels $k' < k$. In this case, we know that $\mathcal{X}^e((\alpha, i_k, \beta)) = \alpha' \times i_k \times \beta$ for some α' . Since $p' = \text{node}(\alpha')$ might be different from p , e may still cause a change of state in the level- k CTMC, even though e is independent of level k . If $p' \neq p$, the level- k CTMC contains arcs from state (p, i_k) to state (p', i_k) for all $(p, i_k) \in \mathcal{M}_k$. The rate of each arc is simply $\mathbf{A}_k^e[p, p']$, the rate at which e causes a transition from node p to node p' .

4. ALGORITHMIC DETAILS

Fig. 3 shows the pseudo-code for our approximation algorithm. Procedure $\text{ComputeMC}(k)$, not shown, computes the rates of the level- k CTMC using Eq. 6, which requires matrices $\mathbf{A}_k$ and vectors $\mathbf{b}_k$. These are computed recursively by procedures ComputeAs and ComputeBs in Fig. 3.

4.1 The fixed-point computation

The overall fixed-point iteration is carried out by procedure Solve . At each iteration, Solve calls $\text{SolveLevel}(k)$ for each k after having updated $\mathbf{A}$ and $\mathbf{b}$. $\text{SolveLevel}(k)$ uses an iterative Markov chain solver to compute the stationary distribution π_k of the level- k CTMC given the current value of its transition rate matrix $\mathbf{R}_k$ (we use Gauss-Seidel with under-relaxation). For efficiency, we use the final iterate π_k of the previous fixed point iteration as the next initial guess in the Gauss-Seidel solution at the next fixed-point iteration. By doing so, the number of Gauss-Seidel iterations required to solve the level- k CTMC tends to decrease from one fixed-point iteration to the next. Indeed, we stop the fixed-point iterations when all K CTMC solutions require just one Gauss-Seidel iteration, as this implies that the transition rate matrix $\mathbf{R}_k$ of the level- k CTMC has not changed enough to require a change (above a given relative precision ϵ) in the stationary solution π_k .

Multiple fixed-point iterations are required only when there is a cyclic dependency among the synchronizing events; that is, the local state i_k for submodel k affects a rate in $\mathbf{R}_l$ and, conversely, the local state i_l for submodel l affects a rate in $\mathbf{R}_k$, for $l \neq k$. If the dependence is only one-way and the submodels are ordered accordingly, our algorithm will not require fixed-point iterations (that is, the second time it computes the vectors π_k , it will realize that the old values

ComputeAs(k)	
1: if $k = K$ then	• There is only the root node
2: for each $e \in \mathcal{E}$ do	
3: $\mathbf{A}_k^e \leftarrow [1]$;	
4: Return	
5: for each $e \in \mathcal{E}$ do	
6: $\mathbf{A}_k^e \leftarrow \mathbf{0}$;	• Clear the old $\mathbf{A}$ matrices
7: for each $p \in \mathcal{L}[k+1]$ do	• Nodes p in level $k+1$
8: for each $i_{k+1}, p[i_{k+1}] \neq \text{null}$ do	
9: $\text{adjust} \leftarrow \pi_{k+1}[(p, i_{k+1})] / \text{Pr}\{p[i_{k+1}]\}$;	
10: for each $e \in \mathcal{E}, \mathcal{X}_{k+1}^e(e, i_{k+1}) \neq \text{null}$ do	
11: for each $j_{k+1} \in \mathcal{X}_{k+1}^e(e, i_{k+1})$ do	
12: for each $q \in \mathcal{L}[k+1], \mathbf{A}_{k+1}^e[p, q] \neq 0$ do	
13: if $q[j_{k+1}] \neq \text{null}$ then	
14: $\text{rate} \leftarrow \mathbf{A}_{k+1}^e[p, q] \cdot \mathbf{W}_{k+1}^e[i_k, j_k]$;	
15: $\mathbf{A}_k^e[p[i_k], q[j_k]] \leftarrow \mathbf{A}_k^e[p[i_k], q[j_k]] + \text{rate} \cdot \text{adjust}$;	
ComputeBs(k)	
1: for each $p \in \mathcal{L}[k]$ do	
2: for each $e \in \mathcal{E}$ do	
3: $\mathbf{b}_k^e[p] \leftarrow 0$;	• Clear the $\mathbf{b}$ vector for node p
4: for each $i_k, p[i_k] \neq \text{null}$ do	
5: $\text{adjust} \leftarrow \pi_{k+1}[(p, i_{k+1})] / \text{Pr}\{p\}$;	
6: for each $e \in \mathcal{E}$ such that $\mathcal{X}_k^e(i_k) \neq \text{null}$ do	
7: if $k > 1$ then	
8: $\text{down} \leftarrow \mathbf{b}_{k-1}^e[p[i_k]]$;	
9: else	
10: $\text{down} \leftarrow 1$;	• Terminal case
11: $\mathbf{b}_k^e[p] \leftarrow \mathbf{b}_k^e[p] + \text{adjust} \cdot \text{down} \cdot \lambda_k^e[i_k]$;	
SolveLevel(k)	
1: $\text{ComputeMC}(k)$;	
2: compute probability vector solution of $\pi_k \cdot \mathbf{Q}_k = \mathbf{0}$;	
3: compute probability $\text{Pr}\{p\}$ of each node $p \in \mathcal{L}[k]$;	
Solve()	
1: set up the structures to store $\mathbf{A}$ and $\mathbf{b}$;	
2: initialize local stationary probabilities to some guess;	
3: while not converged do	
4: for $k \leftarrow 1$ to K do	
5: $\text{ComputeBs}(k)$;	
6: for $k \leftarrow K$ down to 1 do	
7: $\text{ComputeAs}(k)$;	
8: $\text{SolveLevel}(k)$;	

Figure 3: Our approximation algorithm.

still represent the stationary solutions, since no $\mathbf{R}_k$ changes after the first update).

4.2 Data structures

For a given level k , we must conceptually represent $|\mathcal{E}|$ matrices $\mathbf{A}_k^e \in \mathbb{R}^{|\mathcal{L}[k]| \times |\mathcal{L}[k]|}$, one for each event e . As these matrices are typically quite sparse, in practice we use a single matrix per level in which only the nonzero elements are stored, with an associated event index. Hence, an entry is a list of pairs (real-value, event-index). Only the rate information changes during the fixed point iterations, so we can build the sparse structure of these matrices once during initialization, and simply adjust the real-value part during the iterations. As discussed in Sect. 3.5, we do not need to store $\mathbf{A}_k^e$ for $k \geq \text{Last}(e)$, since these matrices are identities.

Another possible simplification is to merge the set $\mathcal{E}_k$ of local events for level k into a single “macro event” l_k , with $First(l_k) = Last(l_k) = k$, whose behavior is defined by:

$$\begin{aligned}\mathcal{X}_k^{l_k}(i_k) &= \bigcup_{e \in \mathcal{E}_k} \mathcal{X}_k^e(i_k) \\ \lambda_k^{l_k}[i_k] &= \sum_{e \in \mathcal{E}_k} \lambda_k^e[i_k] \\ Prob_k^{l_k}(i_k, j_k) &= \sum_{e \in \mathcal{E}_k} Prob_k^e(i_k, j_k) \cdot \frac{\lambda_k^e[i_k]}{\lambda_k^{l_k}[i_k]}.\end{aligned}$$

We must also compute and store the value of $\mathbf{b}$ for each (event,node) pair. Our implementation uses a vector of size $|\mathcal{E}|$ per node but, in principle, it could use a vector of size $|\{e \in \mathcal{E} : First(e) > k \geq Last(e)\}|$ for each node at level k , provided we maintain appropriate indexing information. Of course, for each level- k CTMC we also store its stationary probability vector $\pi_k \in \mathbb{R}^{|\mathcal{M}_k|}$, and a vector of size $|\mathcal{L}_k|$ containing the $Pr\{p\}$ for each node p at that level.

5. PRODUCT-FORM MODELS

We now show that our approximation algorithm performs an exact aggregation, thus produces exact results, if the model under study possesses a product-form solution. For simplicity, we limit our proof to the case of a single class product-form closed queueing network with K queues and N customers, where the state space contains $\binom{N+K-1}{N}$

states, $\mathcal{S} = \{\mathbf{i} = (i_K, \dots, i_1) : \sum_{k=1}^K i_k = N\}$. For such a model, there exist a constant G and K functions f_k such that $\forall \mathbf{i} \in \mathcal{S}$, $\pi[\mathbf{i}] = 1/G \cdot \prod_{k=1}^K f_k(i_k)$. Since our only assumption is that $Pr\{i_k|\alpha\} = Pr\{i_k|node(\alpha)\}$ all we have to show is that this assumption is valid for the case of our product-form queueing network.

Observing the decision diagram in Fig. 2(b), it is immediately apparent that the state space $\mathcal{S}$ of any queueing network of the type we consider will have a similar decision diagram, with K levels and $N+1$ nodes per level, except of course at the top level, assuming that each queue is treated as a separate submodel. The meaning of the $N+1$ nodes at level k is clear: starting from the left, the first node corresponds to the fact that all N customers are still to be placed in queues $k, \dots, 1$; the second node corresponds to the fact that one customer is at one of the queues $K, \dots, k+1$, so only $N-1$ customers are still to be placed in queues $k, \dots, 1$; and so on, until the last node, where no customers are left to be placed in queues $k, \dots, 1$. In other words, the common property distinguishing the paths $\alpha = (i_K, \dots, i_{k+1})$ leading to the n^{th} node at level k is that they correspond to having assigned the same number $L = i_K + \dots + i_{k+1}$ of customers to queues $K, \dots, k+1$.

Thus, the meaning of condition $Pr\{i_k|\alpha\} = Pr\{i_k|node(\alpha)\}$ is, in the case of our queueing network, that

$$Pr\{i_k|i_K, \dots, i_{k+1}\} = Pr\{i_k|L = i_K + \dots + i_{k+1}\}.$$

where

$$Pr\{i_k|i_K, \dots, i_{k+1}\} = \frac{\sum_{i_{k-1}, \dots, i_1 : i_{k-1} + \dots + i_1 = N-L-i_k} \frac{1}{G} \prod_{k=1}^K f_k(i_k)}{\sum_{i_k, \dots, i_1 : i_k + \dots + i_1 = N-L} \frac{1}{G} \prod_{k=1}^K f_k(i_k)}$$

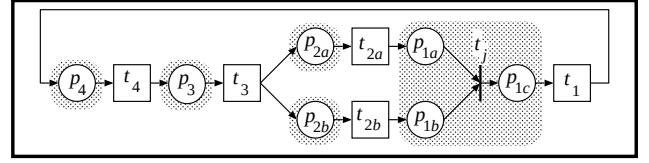


Figure 4: A Fork-join model.

$$\text{and} \quad Pr\{i_k|L\} = \frac{\sum_{i_K, \dots, i_{k+1} : i_K + \dots + i_{k+1} = L} \sum_{i_{k-1}, \dots, i_1 : i_{k-1} + \dots + i_1 = N-L-i_k} \frac{1}{G} \prod_{k=1}^K f_k(i_k)}{\sum_{i_K, \dots, i_{k+1} : i_K + \dots + i_{k+1} = L} \sum_{i_k, \dots, i_1 : i_k + \dots + i_1 = N-L} \frac{1}{G} \prod_{k=1}^K f_k(i_k)}.$$

The two can indeed be shown to be equal with a derivation based on stratification by queue lengths, along the lines of the classical proof of the “flow equivalent server” for product form queueing networks [6].

Concluding this section, we stress that the relevance of the fact that our algorithm gives exact results for this class of models is that it provides us with an additional “sanity check” about the robustness of our approach. Of course, we are not suggesting that the solution of K CTMCs, each with $(N+1)(N+2)/2$ states, except the first and the last, which have only $N+1$ states, in a fixed-point iterative fashion is necessarily a good way to analyze a product-form model.

6. NUMERICAL RESULTS

In this section, we evaluate the proposed approximation approach on a set of experiments. Two small examples are considered initially to observe the effect of synchronization, a fork and join model and a juggling server model, followed by two classical Petri net models: a Kanban [10] and a flexible manufacturing system [12]. All experiments were performed on a Pentium II 400Mhz machine, with 384Mbytes of RAM, with the Linux operating system, and without the use of virtual memory. The exact results used to evaluate our approximation were obtained with SMART [9]. In each case, we report the relative error in aggregated measures, such as the mean number of tokens in a given place or the throughput of transitions. In all cases, CTMCs were solved using Gauss-Seidel with iterations continuing until the relative element-wise difference between subsequent probability vectors was less than 10^{-6} .

6.1 Fork and join model (FJ)

Fig. 4 depicts a Petri net where jobs undergo a certain sequence of activities (transitions t_1 , t_4 and t_3), followed by a parallel execution (transitions t_{2a} and t_{2b}), and a join (transition t_j). All transitions have exponentially distributed delays, except for t_j , which is immediate. The transitions rates of t_4 , t_3 , t_{2a} , t_{2b} , and t_1 are 4, 3, 2, λ , and 1, respectively. There are N jobs in the system, modeled by an initial marking of N for place p_4 .

In the experiments, we vary the value of N and λ , assuming a partition of the net into five components. The size of the state space obviously increases with the number N of circulating jobs: Table 1 lists the size of the state space

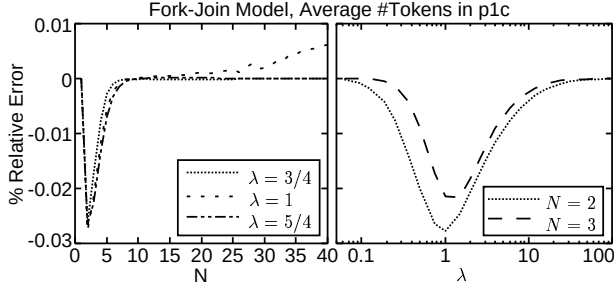


Figure 5: Error for the FJ model of Fig. 4.

N	$ S $	$ \mathcal{M}_5 $	$ \mathcal{M}_4 $	$ \mathcal{M}_3 $	$ \mathcal{M}_2 $	$ \mathcal{M}_1 $
10	1,716	11	66	66	506	121
20	19,481	21	231	231	3,311	411
30	87,296	31	496	496	10,416	961
40	259,161	41	861	861	23,821	1,681

Table 1: State space for the FJ model of Fig. 4.

of the complete model (column “ $|S|$ ”) and the size of the Markov chains used by the approximation for level k (column “ $|\mathcal{M}_k|$ ”), for $k = 5, \dots, 1$. For the largest case, $N = 40$, there is about one order of magnitude between the size of the whole system and that of the largest Markov chain solved by our approximation; much larger savings are realized for larger values of N .

Another interesting measure of complexity is the number of iterations of the fixed point algorithm. For $N = 40$ and $\lambda = 1$, only five fixed-point iterations are required and, for each of them, Table 2 reports the number of iterations performed by the Gauss-Seidel solution of the level- k CTMC. Indeed, the approximation reaches a fixed point in five iterations for all the values of λ we used. In this model, the solution for levels 4 and 5 converges very quickly (as indicated by a single Gauss Seidel iteration after the first step).

Fig. 5 shows the relative error against the number of jobs N and the rate λ for the computation of the average number of tokens in place p_{1c} . Note that the approximation is quite accurate for this model: usually within 0.01%. Relative errors for other places were similar or better, and the *maximum observed throughput error for transitions was 0.02%*.

6.2 Load dependent service

Fig. 6 depicts a Petri net where two stations share S servers. Jobs in the system progress cyclically through four activities (transitions t_4, t_3, t_2 , and t_1), and S servers move between transitions t_2 and t_1 , through transitions i_{21} and i_{12} . All delays are exponentially distributed and the transitions rates are for t_4 : 1, single server, for t_3 : 1, infinite server, for t_2 : 2, infinite server, for t_1 : λ , infinite server, and for i_{21} and for i_{12} : 10, single server. The infinite server semantics for transitions t_1, t_2 , and t_3 makes their rates depend on the enabling degree, and therefore on the marking of the input places. Place p_4 and s_2 are initialized with N jobs and S servers, respectively

In the experiments, we vary the values of N, S , and λ , assuming a partition of the net into three components. Again, the size of the state space is not very large, and it is not

fixed pt. it.	GSD it. level 5	GSD it. level 4	GSD it. level 3	GSD it. level 2	GSD it. level 1
1	110	293	526	3,082	1,965
2	90	161	230	1	1
3	62	86	1	1	1
4	20	1	1	1	1
5	1	1	1	1	1

Table 2: Iterations for the FJ model, $\lambda = 1, N = 40$.

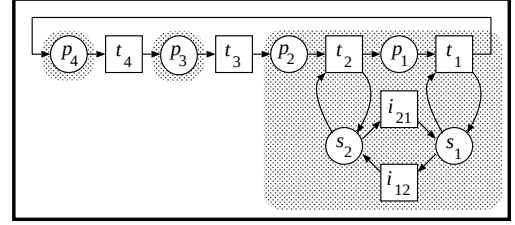


Figure 6: Servers moving between two stations.

reported here since it does not show any significant difference with respect to the fork and join case. Moreover, in all experiments performed (varying N from 1 to 40, S from 1 to 10, with 10 different values for λ), the approximation converges in four iterations, with a pattern very similar to the fork and join case.

Fig. 7 shows the relative error against the number of jobs N and the rate λ for the computation of the average number of tokens in place p_4 . The highest error occurs with a single server when the service rate is high. In general, we have observed that our approximation tends to have a high error due to synchronizing events whose rates can change significantly; this tends to occur when the event of a synchronizing event has a relatively high, state-dependent rate. The maximum observed throughput error for transitions was 2.3% for this model.

6.3 A Kanban system

Fig. 8 depicts the Petri net of a Kanban system [10] composed of four stations; indices in the place and transition names identify the station. A part enters a station only if a Kanban “ticket” is available (places p_{kan_i}); if the work performed is “ok” the part can move to the next station, otherwise we must “fix” it. A part that passes station 1 forks into two parts, one for station 2 and one for station 3; the two parts then join and move to station 4. All transitions have exponentially distributed delays. The complexity

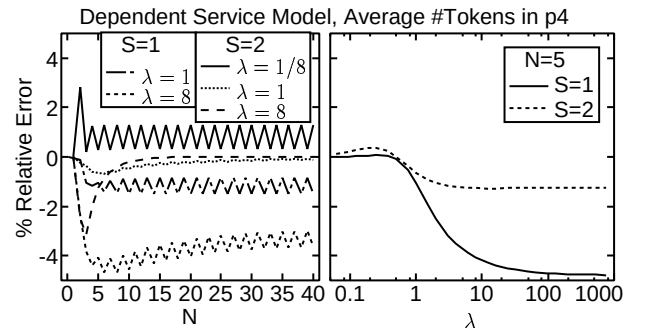


Figure 7: Error for the net of Fig. 6.

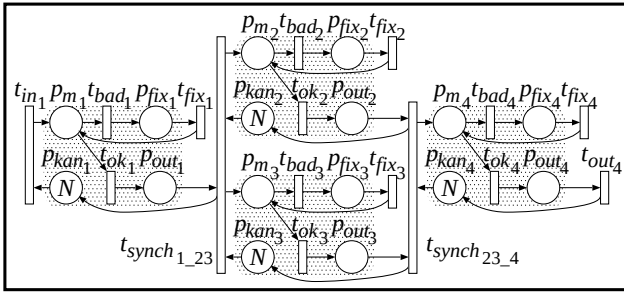


Figure 8: The Kanban model.

N	S	M_k	CPU
5	2.5×10^6	56	0.5
10	1.0×10^9	286	5.0
40	9.9×10^{14}	12,341	1,262.7
60	7.2×10^{16}	39,711	7,787.9

Table 3: State space for the Kanban of Fig. 8.

of the model grows rapidly with the number N of Kanban tokens at each station. This shows how the method behaves, in terms of memory and time, on models with very large state spaces.

In the experiments, we vary the value of N and consider a partition with a single station per component (4 levels). Table 3 reports the size of the state space, CTMC size for any level k (all Markov chains have the same number of states), and total CPU time for solution (including state space generation), for various values of N . For the largest case ($N = 60$), Table 4 shows the number of Gauss-Seidel iterations at each of the required seven fixed point iterations. An exact solution is possible only for N up to 5: for the cases in which the comparison is possible the *relative error on the throughput of transitions is always less than 1%*.

6.4 A flexible manufacturing system (FMS)

Fig. 9 depicts a Petri net of a FMS system [12] with three machines (places M_1 , M_2 , and M_3). Four types of parts are present, and are modeled by the four places P_1 , P_2 , P_3 , and P_{12} : only the first three are initialized with N tokens, since parts of the last type come from the combination of parts of type 1 and 2. Also in this case the complexity of the model grows rapidly with N . The transition rates used are similar to those in [12], except immediate transitions are replaced with fast transitions, to allow us to partition the model into 19 components.

Table 5 reports the size of the state space, the range of CTMC sizes (since there are 19 CTMCs, we only list the size

fixed pt. it.	GSD it. level 4	GSD it. level 3	GSD it. level 2	GSD it. level 1
1	1,462	1,923	1,819	1,698
2	830	1,152	927	814
3	461	493	40	3
4	1	20	12	5
5	1	6	4	1
6	1	2	1	1
7	1	1	1	1

Table 4: Iterations for the Kanban, $N = 60$.

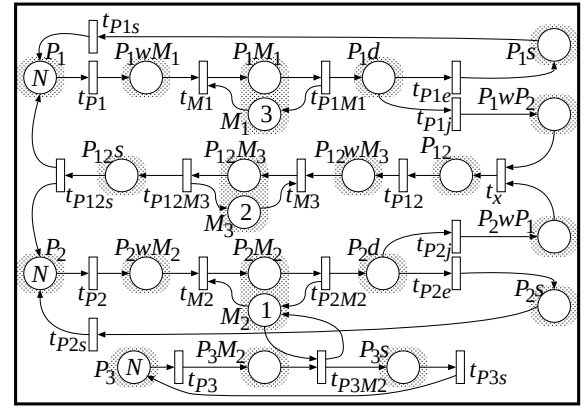


Figure 9: The FMS model.

N	S	$\min_k M_k$	$\max_k M_k$	CPU
6	1.5×10^7	7	714	33.6
12	1.7×10^{10}	13	7,280	673.5
24	5.2×10^{13}	25	90,675	14,640.4

Table 5: State space for the FMS of Fig. 9.

of the smallest and of the largest ones), and the total CPU time required for solution, for the FMS as a function of N . For $N = 12$, 112 fixed point iterations are required. Table 6 reports the minimum and maximum number of Gauss-Seidel iterations required by the 19 CTMCs. An exact solution is possible only for N up to 6. The maximum observed relative error on the throughput of transitions was 14% for the case $N = 1$ and 4.6% for the case $N = 6$.

7. CONCLUSION

We presented an approximation algorithm for the computation of stationary measures in a large structured Markov model described as the composition of K submodels. By using decision diagrams and Kronecker operators, we encode the exact state space and transition rate matrix of the model very efficiently. The algorithm uses this information to build K Markov chains corresponding to an approximate aggregation of the exact process. This allows us to obtain accurate approximations for large models whose exact numerical solution would be otherwise infeasible. As a special case, we have also shown how, for models possessing a product-form solution, our algorithm provides exact results.

A benefit of using the exact state space description is that we assign nonzero probability only to truly reachable states. In contrast, other decomposition approximations may assign a nonzero probability to an unreachable state. In our experience, our approach produces more accurate results but tends to be more expensive (i.e., requires the solution of larger CTMCs) than other model-based decompositions. Our approach is ultimately limited by the sizes of the CTMCs to be solved at each level, which depend on the decision diagram encoding of the state space.

We illustrated the effectiveness of the proposed algorithm through several representative models, including queueing networks with fork-and-join behavior or server movement, and stochastic Petri net models of factory automation systems with general synchronizations and interactions among

fixed pt. it.	GSD it. (min)	GSD it. (max)
1	30	2,000
2	20	465
10	24	109
20	14	75
50	1	35
112	1	1

Table 6: Iterations for the FMS, $N = 12$.

the subsystems. For the set of experiments we performed (of which we report only a subset for lack of space), the results are quite accurate and they are obtained in a modest amount of time (usually within a few hours).

An additional strength of our approach is that experimenting with different decompositions of the overall model simply involves redefining a partition of the queues or places in the model.

8. REFERENCES

- [1] R. I. Bahar, E. A. Frohm, C. M. Gaona, G. D. Hachtel, E. Maciui, A. Pardo, and F. Somenzi. Algebraic decision diagrams and their applications. *Formal methods in system design*, 10(2/3):171–206, Apr. 1997.
- [2] R. E. Bryant. Symbolic boolean manipulation with ordered binary-decision diagrams. *ACM Comp. Surv.*, 24(3):393–318, 1992.
- [3] P. Buchholz. Numerical solution methods based on structured descriptions of Markovian models. In G. Balbo and G. Serazzi, editors, *Computer performance evaluation*, pages 251–267. Elsevier Science Publishers B.V. (North-Holland), 1991.
- [4] P. Buchholz, G. Ciardo, S. Donatelli, and P. Kemper. Complexity of memory-efficient Kronecker operations with applications to the solution of Markov models. *INFORMS J. Comp.* To appear.
- [5] J. Campos, M. Silva, and S. Donatelli. Structured solution of stochastic DSSP systems. In *Proc. 7th Int. Workshop on Petri Nets and Performance Models (PNPM’97)*, pages 91–100, St. Malo, France, June 1997. IEEE Comp. Soc. Press.
- [6] K. M. Chandy, U. Herzog, and L. Woo. Parametric analysis of queueing networks. *IBM J. Res. and Dev.*, 19:36–42, Jan. 1975.
- [7] H. Choi and K. S. Trivedi. Approximate performance models of polling systems using stochastic Petri nets. In *Proc. IEEE INFOCOM 1992*, pages 2306–2314, Florence, Italy, May 1992.
- [8] G. Ciardo, G. Luetzgen, and R. Siminiceanu. Efficient symbolic state-space construction for asynchronous systems. In M. Nielson and D. Simpson, editors, *Application and Theory of Petri Nets 2000 (Proc. 21th Int. Conf. on Applications and Theory of Petri Nets, Aarhus, Denmark)*. Springer-Verlag, June 2000. To appear.
- [9] G. Ciardo and A. S. Miner. SMART: Simulation and Markovian Analyzer for Reliability and Timing. In *Proc. IEEE International Computer Performance and Dependability Symposium (IPDS’96)*, page 60, Urbana-Champaign, IL, USA, Sept. 1996. IEEE Comp. Soc. Press.
- [10] G. Ciardo and A. S. Miner. Storage alternatives for large structured state spaces. In R. Marie, B. Plateau, M. Calzarossa, and G. Rubino, editors, *Proc. 9th Int. Conf. on Modelling Techniques and Tools for Computer Performance Evaluation*, Lecture Notes in Computer Science 1245, pages 44–57, St. Malo, France, June 1997. Springer-Verlag.
- [11] G. Ciardo and A. S. Miner. A data structure for the efficient Kronecker solution of GSPNs. In P. Buchholz, editor, *Proc. 8th Int. Workshop on Petri Nets and Performance Models (PNPM’99)*, pages 22–31, Zaragoza, Spain, Sept. 1999. IEEE Comp. Soc. Press.
- [12] G. Ciardo and K. S. Trivedi. A decomposition approach for stochastic reward net models. *Perf. Eval.*, 18(1):37–59, 1993.
- [13] S. Donatelli. Superposed Stochastic Automata: a class of stochastic Petri nets with parallel solution and distributed state space. *Perf. Eval.*, 18:21–26, 1993.
- [14] P. Fernandes, B. Plateau, and W. J. Stewart. Efficient descriptor-vector multiplication in stochastic automata networks. *Journal of the ACM*, 45(3):381–414, 1998.
- [15] P. Heidelberger and K. S. Trivedi. Queueing network models for parallel processing with asynchronous tasks. *IEEE Trans. Comp.*, C-31(11):1099–1109, Nov. 1982.
- [16] P. Heidelberger and K. S. Trivedi. Analytic queueing models for programs with internal concurrency. *IEEE Trans. Comp.*, C-32(1):73–82, Jan. 1983.
- [17] O. C. Ibe, R. C. Howe, and K. S. Trivedi. Approximate availability analysis of VAXcluster systems. *IEEE Trans. Rel.*, 38(1):146–152, Apr. 1989.
- [18] V. Mainkar and K. S. Trivedi. Fixed point iteration using Stochastic Reward Nets. In *Proc. 6th Int. Workshop on Petri Nets and Performance Models (PNPM’95)*, pages 21–30, Durham, NC, Oct. 1995. IEEE Comp. Soc. Press.
- [19] A. S. Miner and G. Ciardo. Efficient reachability set generation and storage using decision diagrams. In H. Kleijn and S. Donatelli, editors, *Application and Theory of Petri Nets 1999 (Proc. 20th Int. Conf. on Applications and Theory of Petri Nets, Williamsburg, VA, USA)*, Lecture Notes in Computer Science 1639, pages 6–25. Springer-Verlag, June 1999.
- [20] E. Pastor, O. Roig, J. Cortadella, and R. Badia. Petri net analysis using boolean manipulation. In R. Valette, editor, *Application and Theory of Petri Nets 1994, (Proc. 15th Int. Conf. on Applications and Theory of Petri Nets, Zaragoza, Spain)*, Lecture Notes in Computer Science 815, pages 416–435. Springer-Verlag, June 1994.
- [21] B. Plateau. On the stochastic structure of parallelism and synchronisation models for distributed algorithms. In *Proc. 1985 ACM SIGMETRICS Conf. on Measurement and Modeling of Computer Systems*, pages 147–153, Austin, TX, USA, May 1985.
- [22] K. C. Sevcik. Priority scheduling disciplines in queueing network models of computer systems. In *1977 IFIP Congr. Proc.*, 1977.
- [23] A. Srinivasan, T. Kam, S. Malik, and R. K. Brayton. Algorithms for discrete function manipulation. In *International Conference on CAD*, pages 92–95. IEEE Computer Society, 1990.
- [24] W. J. Stewart, K. Atif, and B. Plateau. The numerical solution of stochastic automata networks. *Europ. J. of Oper. Res.*, 86:503–525, 1995.
- [25] Y. Takahashi. Aggregate approximation for acyclic queueing networks with communication blocking. In H. G. Perros and T. Altiok, editors, *Queueing Networks with Blocking*, pages 33–47. Elsevier Science Publishers B.V., 1989.
- [26] L. Tomek and K. S. Trivedi. Fixed-Point Iteration in Availability Modeling. In M. Dal Cin, editor, *Informatik-Fachberichte, Vol. 91: Fehlertolerierende Rechensysteme*, pages 229–240. Springer-Verlag, 1991.
- [27] C. M. Woodside. Throughput calculation for basic Stochastic Rendezvous Networks. *Perf. Eval.*, 9:143–160, 1988/89.